

Roadmap Strategica e Architettura

Tecnica per "SaluteTwin": Sviluppo MVP

Android Native (Visione 2026)

Data: 18 Gennaio 2026

Autore: Senior Technical Project Manager & Lead Software Architect

Stato: Documento Esecutivo Finale

Classificazione: Confidenziale / Uso Interno

1. Executive Summary e Visione Strategica

1.1 Il Paradigma del "Gemello Digitale" nel Mercato Italiano

L'analisi del panorama mHealth (mobile Health) in Italia per il triennio 2025-2030 rivela una dicotomia fondamentale: da un lato, una crescente adozione di dispositivi wearable di fascia media (trainata da brand come Xiaomi, Amazfit e Fitbit), dall'altro, una saturazione di applicazioni che si limitano alla mera visualizzazione di dati grezzi senza offrire un reale valore cognitivo o emotivo all'utente. Il progetto "SaluteTwin" nasce per colmare questa lacuna, posizionandosi non come l'ennesima dashboard biometrica, ma come un compagno digitale empatico. Basandosi sulle proiezioni che indicano un mercato italiano delle app mHealth prossimo ai 2 miliardi di dollari entro il 2030, con un CAGR del 15,8%¹, l'opportunità risiede nella democratizzazione dell'analisi sanitaria avanzata. L'obiettivo è trasformare i dati frammentati provenienti da hardware eterogeneo ed economico in un'esperienza utente coesa, sfruttando l'Intelligenza Artificiale Generativa per creare un dialogo "friction-free" che aumenti la ritenzione, storicamente il tallone d'Achille delle app Android in questo settore.¹

1.2 Obiettivi del Rapporto e Mandato Tecnico

Il presente documento costituisce la roadmap tecnica e operativa per lo sviluppo del Minimum Viable Product (MVP) di SaluteTwin. In qualità di Senior Technical Project Manager, l'approccio adottato è rigorosamente "Engineering-First". Non si tratta solo di definire cosa costruire, ma *come* costruirlo in modo che l'architettura possa sostenere l'evoluzione futura del prodotto senza accumulare debito tecnico paralizzante.

Le direttive strategiche impongono vincoli precisi che guidano ogni decisione architetturale contenuta in questo rapporto:

- 1. Modularizzazione Feature-Based:** L'applicazione deve essere concepita fin dal primo giorno come un insieme di moduli funzionali indipendenti. Questo non è solo un vezzo stilistico, ma una necessità operativa per gestire un team distribuito e preparare il terreno

per una futura migrazione a Kotlin Multiplatform (KMP), rendendo la logica di business riutilizzabile su iOS o Desktop.²

2. **Scalabilità dei Dati (Local-First):** L'architettura deve prevedere un'astrazione dei dati tramite Repository Pattern che permetta di operare inizialmente con un database locale (Room), garantendo la transizione trasparente verso un sistema di sincronizzazione cloud o ibrido nelle fasi successive.³
3. **Gestione Asincrona del Team:** Con un modello operativo "1 Lead + 2 Contributor", il flusso di lavoro deve essere asincrono e non bloccante. L'adozione di pratiche come il "Documentation-First" e il Trunk-Based Development con Feature Flags è mandatoria per mantenere alta la velocity senza sacrificare la stabilità.⁴

Questo rapporto dettaglia come queste direttive si traducano in codice, processi e milestone temporali, fornendo al team di sviluppo una guida esaustiva per le prossime 12 settimane.

2. Architettura del Sistema: Specifiche Tecniche 2026

L'ecosistema Android nel 2026 richiede standard di qualità e strutturazione del codice estremamente elevati. L'epoca delle "God Activities" è tramontata da tempo; l'architettura moderna deve essere reattiva, modulare e testabile. La scelta di **Clean Architecture** non è opzionale ma strutturale per garantire la longevità del progetto.

2.1 Strategia di Modularizzazione "Feature-Based"

La modularizzazione è la chiave di volta per la scalabilità del team e del codice. Abbandonando l'approccio monolitico o la semplice divisione per layer (data/domain/ui), adottiamo una modularizzazione verticale basata sulle funzionalità (Feature-Based Modularization).

2.1.1 Razionale Architetture

Nel 2026, la best practice consolidata suggerisce di isolare ogni funzionalità di business in un modulo Gradle separato. Questo offre vantaggi immediati:

- **Tempi di Build Ridotti:** Gradle può sfruttare la build cache e la compilazione incrementale in modo più efficace se i moduli sono disaccoppiati.⁶
- **Separazione delle Responsabilità:** Un developer che lavora sul modulo `:feature:chat` non rischia di rompere accidentalmente il codice in `:feature:onboarding`.
- **Portabilità KMP:** I moduli di puro dominio (business logic) possono essere facilmente convertiti in moduli Kotlin Multiplatform se privi di dipendenze dal framework Android, preparando l'app per un futuro cross-platform.⁷

2.1.2 Grafo delle Dipendenze e Struttura dei Moduli

L'applicazione sarà strutturata secondo il seguente albero di moduli, gestito tramite un `libs.versions.toml` (Version Catalog) per la coerenza delle dipendenze.⁸

Livello Modulo	Nome Modulo	Responsabilità e Contenuto Tecnico	Tipo di Plugin Gradle
Application	<code>:app</code>	Entry point dell'applicazione. Contiene il grafo di navigazione principale (Navigation Host), l'iniezione delle dipendenze root (Hilt/Koin root) e il tema globale. Non contiene logica di business.	<code>com.android.application</code>
Feature	<code>:feature:onboarding</code>	Gestione del flusso di benvenuto, richiesta permessi (Health Connect, Camera, Notifiche) e setup profilo utente.	<code>com.android.library</code>
Feature	<code>:feature:dashboard</code>	Home screen dell'app. Visualizzazione dell'Avatar (Digital Twin), grafici di sintesi (Sleep Debt, Readiness) e card riassuntive.	<code>com.android.library</code>

Feature	:feature:chat	Interfaccia conversazionale (UI Chat). Gestione della history dei messaggi e interazione con l'agente AI.	com.android.library
Feature	:feature:measurement	Modulo dedicato all'acquisizione biometrica tramite fotocamera (CameraX). Contiene la UI del viewfinder e il feedback in tempo reale.	com.android.library
Core (Domain)	:core:domain	Il cuore dell'applicazione. Contiene i Use Cases, le Entities di dominio e le Interfacce dei Repository. Deve essere Pure Kotlin (no Android dependencies) per essere KMP-ready.	kotlin (o kotlin-multiplatform)
Core (Data)	:core:data	Implementazione dei Repository. Gestisce Room Database, DataStore e la logica di coordinamento delle fonti dati (Health Connect vs Local DB).	com.android.library

Core (Network)	:core:network	Client API per comunicare con servizi esterni (LLM Backend, Meteo API). Utilizza Retrofit/Ktor.	com.android.library
Core (System)	:core:camera	Logica di basso livello per l'elaborazione delle immagini (ImageAnalysis) e il calcolo del segnale PPG. Isolato per facilitare i test e la sostituzione.	com.android.library
Core (System)	:core:health	Wrapper per l'integrazione con Health Connect API. Gestisce la complessità dei permessi e della lettura asincrona.	com.android.library
Core (UI)	:core:ui	Design System. Contiene i componenti Jetpack Compose riutilizzabili (Buttons, TextStyles, Theme, Icons) per garantire coerenza visiva.	com.android.library

2.2 Lo Stack Tecnologico: Scelte per il 2026

La selezione tecnologica privilegia la stabilità, la performance e la "future-proofness".

- **Linguaggio:** Kotlin 2.1+. L'uso del compilatore K2 è mandatorio per migliorare le performance di compilazione e abilitare le nuove funzionalità del linguaggio.¹⁰
- **UI Framework: Jetpack Compose.** È lo standard de facto. L'approccio dichiarativo si sposa perfettamente con il pattern Unidirectional Data Flow (UDF).
- **Dependency Injection (DI): Hilt.** Sebbene Koin sia popolare nella community KMP, Hilt offre una sicurezza a compile-time (tramite Dagger) superiore, riducendo i rischi di crash a runtime in un team dove il livello di esperienza può variare.¹¹ Tuttavia, l'architettura disaccoppiata (interfacce in :core:domain) permetterà di migrare a Koin in futuro se si deciderà di portare l'app su iOS, dove Hilt non è supportato.
- **Build System:** Gradle con **Convention Plugins.** Invece di duplicare la configurazione (es. versioni Java, flag di compilazione) in ogni build.gradle, creeremo plugin personalizzati (es. com.salutetwin.android.library, com.salutetwin.android.feature) nella cartella build-logic. Questo standardizza i moduli e semplifica l'onboarding dei collaboratori.¹³

2.3 Implementazione del Repository Pattern e Scalabilità Dati

Il requisito di scalabilità "Local-to-Cloud" viene soddisfatto implementando rigorosamente il pattern Repository come "Single Source of Truth".

2.3.1 Design del Repository

Ogni Repository espone solo dati di dominio (non entità DB o DTO di rete) tramite flussi reattivi (kotlinx.coroutines.Flow).

Kotlin

```
// In :core:domain (Pura Kotlin)
interface ReadinessRepository {
    // Osserva i cambiamenti (Reattivo)
    fun getDailyReadiness(): Flow<ReadinessStatus>

    // Richiede un aggiornamento forzato (es. pull da Health Connect)
    suspend fun refreshData(): Result<Unit>
}
```

2.3.2 Implementazione "Offline-First"

Nel modulo `:core:data`, l'implementazione concreta orchestrerà le fonti dati.

- **Fase 1 (MVP):** Il Repository legge/scrive esclusivamente su **Room Database**. Quando arrivano dati da Health Connect, vengono salvati in Room, e Room notifica la UI tramite il Flow.
- **Fase 2 (Cloud):** Si aggiungerà una RemoteDataSource. Il Repository tenterà di sincronizzare Room con il Cloud. La UI continuerà a osservare Room, rimanendo agnostica rispetto alla sincronizzazione. Questo disaccoppiamento è cruciale.³

2.3.3 Strategia per gli ID e la Sincronizzazione

Per mitigare il rischio di conflitti futuri durante la sincronizzazione cloud, **non utilizzeremo ID autoincrementali** (1, 2, 3...) nel database locale.

- **Soluzione:** Utilizzo di **UUID** (o ULID per l'ordinamento temporale) generati dal client come chiavi primarie. Questo permette di creare dati offline che possono essere inviati al server in un secondo momento senza collisioni di ID.³

3. Analisi dei Requisiti MVP e Prioritizzazione (MoSCoW)

Basandosi sull'analisi di mercato ¹ e sulla fattibilità tecnica ¹, abbiamo distillato le funzionalità essenziali per l'MVP. L'obiettivo è validare l'ipotesi di valore (l'empatia del Digital Twin) minimizzando lo sforzo di sviluppo.

3.1 Must Have (Critico per il lancio)

Queste funzionalità costituiscono il nucleo non negoziabile del prodotto. Senza di esse, l'app non assolve alla sua promessa di valore.

1. **Integrazione Health Connect (Read-Only):**
 - *Descrizione:* L'app deve agire come un hub, leggendo dati da Xiaomi/Amazfit/Google Fit tramite le API di Health Connect.¹⁵
 - *Dati Critici:* Passi giornalieri, Durata del Sonno (Totale, Profondo, REM), Frequenza Cardiaca a riposo.
 - *Vincolo Tecnico:* Gestione dei permessi granulari e della latenza di sync.
2. **Acquisizione HRV via Camera (PPG):**
 - *Descrizione:* Poiché i braccialetti economici non forniscono l'HRV (rMSSD) grezzo affidabile, l'app deve misurarlo usando la fotocamera e il flash dello smartphone.¹
 - *Tecnologia:* CameraX ImageAnalysis per elaborare i frame YUV e rilevare le variazioni di intensità del rosso.¹⁶
3. **Algoritmo "Digital Twin Brain" (Locale):**
 - *Descrizione:* Motore logico che aggrega i dati per calcolare due metriche proprietarie: *Sleep Debt* (debito di sonno cumulativo) e *Daily Readiness* (prontezza

giornaliera basata su HRV e sonno).¹

- *Output*: Uno stato semplice (es. "Batteria al 40%").

4. **Interfaccia Chat Empatica:**

- *Descrizione*: UI basata su chat dove l'utente riceve insight. Invece di grafici, l'IA (simulata o via API) invia messaggi come "Ho notato che hai dormito poco, prenditela comoda oggi".¹

5. **Persistenza Locale (Room & DataStore):**

- *Descrizione*: Salvataggio storico delle misurazioni e dello stato della chat per funzionamento offline.

3.2 Should Have (Alta Priorità)

Funzionalità importanti che dovrebbero essere incluse se la velocity del team lo consente, altrimenti rilasciate nella prima patch post-lancio.

1. **Gamification Visiva (Avatar)**: Un'icona o un'immagine del "Gemello" che cambia espressione o stato in base alla Readiness (es. avatar assonnato se Sleep Debt è alto). Aumenta l'engagement emotivo.¹
2. **Notifiche Contestuali Intelligenti**: Trigger locali che inviano una notifica al mattino ("Il tuo report è pronto") o alla sera ("Ora di rallentare").
3. **Analisi Contesto Ambientale**: Integrazione API Meteo semplice per correlare dolori/umore al tempo atmosferico.¹

3.3 Could Have (Desiderabile)

Funzionalità che aggiungono valore ma non sono critiche per la validazione iniziale.

1. **Export PDF per Medico**: Generazione di un report statico.
2. **Fenotipizzazione Digitale (UsageStats)**: Analisi del tempo speso sui social media per correlarlo allo stress. Complesso per via dei permessi speciali richiesti (PACKAGE_USAGE_STATS).¹⁷
3. **Supporto Temi Dinamici**: Adattamento completo ai colori di sistema (Material You).

3.4 Won't Have (Escluso dall'MVP)

Esplicitamente rimossi per ridurre rischi e tempi.

1. **Sincronizzazione Cloud Multi-Device**: L'app sarà "Local-Only". Il backup cloud è complesso e costoso da mantenere all'inizio.
 2. **LLM On-Device (Gemini Nano)**: Sebbene promettente, la compatibilità è limitata ai device top di gamma (Pixel 8/9, Galaxy S24).¹⁸ L'MVP userà API Cloud per garantire supporto alla fascia media.
 3. **Diagnostica Medica**: Nessun claim diagnostico per evitare la classificazione come Dispositivo Medico di Classe IIa (MDR).¹⁹
-

4. Deep Dive Tecnico: Soluzioni alle Sfide Critiche

4.1 La Sfida della Frammentazione Hardware: CameraX e PPG

L'acquisizione del segnale PPG (Photoplethysmography) è la feature più rischiosa tecnicamente. Dobbiamo rilevare impercettibili variazioni di colore della pelle causate dal battito cardiaco.

- **Problema:** Gli algoritmi di post-processing delle fotocamere moderne (HDR, correzione colore, auto-esposizione) tentano di "correggere" queste variazioni, distruggendo il segnale utile. Inoltre, ogni produttore implementa le API Camera2 in modo diverso.²⁰
- **Soluzione Architettureale 2026:** Utilizzo della libreria **CameraX** con l'estensione **Camera2Interop**.
 - CameraX gestisce la complessità del ciclo di vita e della superficie di anteprima meglio di Camera2 puro.
 - L'ImageAnalysis UseCase ci fornisce accesso diretto ai buffer YUV.
 - **Cruciale:** Utilizzeremo Camera2Interop.Extender per disabilitare l'auto-esposizione (CONTROL_AE_MODE_OFF) e fissare la durata del frame e la sensibilità ISO su valori manuali. Questo previene che la luminosità dell'immagine fluttui automaticamente, garantendo che le variazioni di luce siano dovute solo al flusso sanguigno.²²
 - **Fallback:** Implementeremo una logica di "Quality Check". Se la varianza della luminosità media è troppo alta (dito che si muove) o il frame rate scende sotto i 30fps, la misurazione viene invalidata immediatamente con feedback all'utente.

4.2 Integrazione Health Connect e Latenza

SaluteTwin dipende dai dati di terze parti (es. app Zepp Life per Mi Band).

- **Il Problema della Latenza:** Health Connect non è un bus in tempo reale. Le app come Zepp Life scrivono i dati periodicamente (spesso solo quando l'utente apre l'app Zepp). Inoltre, Android 14+ introduce limitazioni severe sulle letture in background per risparmiare batteria.²³
- **Strategia di Mitigazione:**
 1. **Foreground Sync Aggressivo:** Implementiamo un WorkManager che viene triggerato *immediatamente* ogni volta che l'app SaluteTwin viene portata in primo piano (ON_RESUME). Questo massimizza la probabilità di avere dati freschi quando l'utente interagisce.
 2. **UI "Honest":** L'interfaccia mostrerà esplicitamente l'orario dell'ultimo dato ("Dati del sonno aggiornati alle 08:30").
 3. **Algoritmo Resiliente:** Se mancano i dati del sonno della notte scorsa (es. sync non avvenuto), l'algoritmo non deve crashare o mostrare zero. Deve entrare in uno stato "In attesa di dati" o usare una stima basata sulla media storica con un chiaro indicatore visivo di "Dato Stimato".

4.3 Gestione Privacy e MDR (Medical Device Regulation)

Dato il contesto italiano ed europeo, la compliance è critica.

- **Posizionamento:** L'app si dichiara strumento di "Benessere" (Lifestyle).
 - **Implementazione Tecnica:**
 - **Guardrails LLM:** Il prompt di sistema inviato all'AI deve contenere istruzioni rigide: "Non fornire mai diagnosi mediche. Se i valori sono anomali, suggerisci di consultare un medico, ma non etichettare la condizione (es. non dire 'hai un'aritmia')".¹
 - **Privacy Dati:** Tutti i dati sensibili rimangono nel DB Room locale. L'invio all'API LLM (se cloud) deve essere pseudonimizzato (solo i valori numerici, senza nome o identificativi personali).
-

5. Piano di Delegabilità e Organizzazione del Team (Modello 1+2)

Gestire 2 collaboratori saltuari richiede un framework rigido per evitare che il Lead passi tutto il tempo a fare merge conflict resolution o a spiegare task.

5.1 Flusso di Lavoro "Documentation-First"

Prima di scrivere codice, si scrive documentazione.

- **ADR (Architecture Decision Records):** Ogni decisione tecnica significativa (es. "Usare Hilt invece di Koin", "Schema DB Room") viene documentata in un file Markdown nel repo (/docs/adr/001-use-hilt.md). Questo permette ai collaboratori di capire il *perché* delle scelte senza chiedere al Lead.²⁴
- **Specifiche delle Issue:** I ticket (Jira/GitHub) non devono essere vaghi. Devono contenere: "Input", "Output atteso", "Moduli coinvolti" e "Link al Design Figma".

5.2 Git Strategy: Trunk-Based Development Scalato

Per un team asincrono, i branch a lunga vita ("Feature Branch" che durano settimane) sono veleno.

- **La Regola:** I branch vivono al massimo 24-48 ore.
- **Feature Flags:** Come facciamo a mergiare codice incompleto nel main senza rompere l'app? Usiamo i Feature Flags.
 - Esempio: Una nuova UI della Dashboard viene sviluppata dietro un flag if (FeatureFlags.NEW_DASHBOARD_ENABLED). Il codice viene mergiato giornalmente nel main, ma l'utente (e gli altri dev) vedono ancora la vecchia dashboard finché la feature non è completa.²⁵
 - Questo elimina il "Merge Hell" alla fine dello sprint.

5.3 Assegnazione dei Compiti (Delegation Plan)

Ruolo	Focus & Responsabilità	Moduli Assegnati	Tipologia Task
Lead Engineer	Architettura & Core Logic	:core:domain, :core:data, :core:camera	Definizione interfacce, setup Hilt/Gradle, Algoritmi complessi (Signal Processing), Code Review, Gestione Rilasci.
Contributor A	UI & Feature Implementation	:feature:onboarding, :feature:dashboard, :feature:chat, :core:ui	Implementazione Pixel-Perfect delle schermate Compose, creazione componenti Design System, binding dei ViewModel (pre-impostati dal Lead).
Contributor B	Integration & Testing	:core:health-connect, :core:network, Unit Tests	Integrazione API terze parti, gestione permessi, scrittura unit test per la business logic, gestione stringhe e localizzazione.

6. Roadmap Temporale: 12 Settimane (6 Sprint)

Stima Totale: ~600-700 ore uomo (Lead Full-time + 2 Collaboratori Part-time).

Sprint 1: Le Fondamenta (Settimane 1-2)

- **Obiettivo:** Setup del progetto e infrastruttura di base funzionante (Hello World)

modulare).

- **Lead:** Setup Gradle Convention Plugins, Configurazione Hilt, Navigazione base, ADR iniziali.
- **Collab A:** Creazione :core:ui (Colori, Font, Componenti base Material3).
- **Collab B:** Setup infrastruttura CI (GitHub Actions per Build e Lint), setup testing libraries.

Sprint 2: Dati e Persistenza (Settimane 3-4)

- **Obiettivo:** L'app può leggere dati da Health Connect e salvarli localmente.
- **Lead:** Definizione Entità Room e Interfacce Repository in :core:domain.
- **Collab A:** UI Onboarding (schermate statiche) e flusso richiesta permessi.
- **Collab B:** Implementazione :core:health-connect. Lettura passi e sonno. Mapping dei dati nel DB locale.

Sprint 3: Il Sensore e l'Algoritmo (Settimane 5-6)

- **Obiettivo:** Acquisizione PPG funzionante e calcolo Readiness base.
- **Lead:** Sviluppo algoritmo Sleep Debt e Readiness. Setup CameraX analyzer (logica grezza).
- **Collab A:** UI Dashboard (scheletro). Visualizzazione dati dummy nei grafici.
- **Collab B:** Integrazione logica DataStore per salvare le preferenze utente (es. orario sveglia).

Sprint 4: L'Anima del Gemello (Settimane 7-8)

- **Obiettivo:** Interfaccia Chat funzionante e connessa all'AI (Mock/API).
- **Lead:** Ottimizzazione segnale PPG (filtri frequenza). Integrazione reale CameraX -> Algoritmo.
- **Collab A:** UI Chat completa (:feature:chat). Animazioni messaggi, stati di loading.
- **Collab B:** Implementazione client API per LLM (OpenAI/Anthropic/Google Vertex) in :core:network.

Sprint 5: Integrazione e Intelligenza (Settimane 9-10)

- **Obiettivo:** Il ciclo completo funziona: Misurazione -> Analisi -> Chat.
- **Lead:** "Cablaggio" finale. Prompt Engineering: raffinare i prompt di sistema per l'empatia.
- **Collab A:** Polish UI/UX. Dark Mode, transizioni, gestione stati di errore (Empty States).
- **Collab B:** Implementazione notifiche locali. Test esaustivi sui repository.

Sprint 6: Stabilizzazione e Pre-Launch (Settimane 11-12)

- **Obiettivo:** App pronta per il rilascio in Alpha/Beta.
- **Lead:** Analisi performance (Battery Historian, Profiler), Leak detection. Review conformità MDR.
- **Collab A:** Bug fixing UI su device diversi (tablet, schermi piccoli).

- **Collab B:** Scrittura documentazione utente finale (FAQ, Privacy Policy in-app). Preparazione build release (ProGuard/R8).
-

7. Risk Management

Rischio 1: Qualità del Dato PPG su Device Low-End

- **Rischio:** Su un dispositivo da 100€, la fotocamera potrebbe avere un frame rate instabile, rendendo il calcolo dell'HRV inaffidabile.
- **Mitigazione:** Implementare un sistema di "Device Grading". Durante il primo utilizzo, l'app esegue un benchmark della camera. Se il device non supporta i requisiti minimi (es. blocco esposizione), la feature PPG viene disabilitata via software (o mostrato un avviso), ripiegando sui dati Health Connect (meno precisi ma sicuri).¹

Rischio 2: "Process Death" e Perdita di Stato

- **Rischio:** L'utente inizia una misurazione, riceve una chiamata, Android uccide l'app per liberare memoria. Al ritorno, lo stato è perso.
- **Mitigazione:** Utilizzo rigoroso di `SavedStateHandle` nei `ViewModel` per salvare lo stato UI critico. Persistenza immediata su Room di ogni step completato (pattern "Save early, save often").²⁷

Rischio 3: Blocco dello Sviluppo per Dipendenze Esterne

- **Rischio:** Il Contributor B è in ritardo sull'integrazione API, bloccando il Contributor A che deve fare la UI della Chat.
 - **Mitigazione:** Utilizzo di **Interfacce e Mock Repository**. Il Contributor A lavora contro un `FakeChatRepository` che restituisce dati finti istantanei. L'implementazione reale (`RealChatRepository`) viene iniettata tramite Hilt solo quando è pronta e testata. Questo disaccoppia completamente i tempi di sviluppo.¹⁴
-

8. Conclusioni

Questa roadmap non descrive solo la creazione di un'app, ma la fondazione di una piattaforma tecnologica scalabile. L'investimento iniziale nella modularizzazione e nella Clean Architecture, sebbene oneroso nelle prime 2 settimane, pagherà dividendi esponenziali riducendo il debito tecnico e facilitando il lavoro parallelo del team. Con l'adozione di Health Connect come ponte universale e di CameraX per l'indipendenza hardware, "SaluteTwin" è posizionata per essere tecnologicamente robusta quanto è commercialmente promettente nel mercato italiano del 2026.

Bibliografia

1. Analisi di Mercato App Salute Freemium.pdf
2. Guide to Android app modularization | App architecture - Android Developers, accesso eseguito il giorno gennaio 18, 2026, <https://developer.android.com/topic/modularization>
3. The Complete Guide to Offline-First Architecture in Android | by Akshay Nandwana | Dec, 2025 | ProAndroidDev, accesso eseguito il giorno gennaio 18, 2026, <https://proandroiddev.com/the-complete-guide-to-offline-first-architecture-in-android-490d5e9d27ea>
4. How to Improve Documentation Practices in Engineering Teams - Chris Wahl, accesso eseguito il giorno gennaio 18, 2026, <https://wahlnetwork.com/2024/03/12/how-to-improve-documentation-practices-in-engineering-teams/>
5. Use Feature Flags for trunk-based development, accesso eseguito il giorno gennaio 18, 2026, <https://developer.harness.io/docs/feature-flags/get-started/trunk-based-development/>
6. From Monolith to Modules: Modernizing Your Android App Architecture | by Vaibhav Shakya | Mr Neo | ProAndroidDev, accesso eseguito il giorno gennaio 18, 2026, <https://proandroiddev.com/from-monolith-to-modules-modernizing-your-android-app-architecture-2f99338e8d27>
7. Is Kotlin Multiplatform production-ready in 2026? - Volpis, accesso eseguito il giorno gennaio 18, 2026, <https://volpis.com/blog/is-kotlin-multiplatform-production-ready/>
8. Gradle Version Catalog in Android: A Complete Guide | by amol pawar | softAai Blogs, accesso eseguito il giorno gennaio 18, 2026, <https://medium.com/softaai-blogs/gradle-version-catalog-in-android-a-complete-guide-b437cdd28cc9>
9. Gradle best practices | Kotlin Documentation, accesso eseguito il giorno gennaio 18, 2026, <https://kotlinlang.org/docs/gradle-best-practices.html>
10. Kotlin 2.1 and Beyond: Why the Modern Multiplatform Standard is the Strategic Choice for 2026 | BolderApps Blog, accesso eseguito il giorno gennaio 18, 2026, <https://www.bolderapps.com/blog-posts/kotlin-2-1-and-beyond-why-the-modern-multiplatform-standard-is-the-strategic-choice-for-2026>
11. Dependency Injection Wars: Koin vs Hilt in Modern Android Projects - Medium, accesso eseguito il giorno gennaio 18, 2026, <https://medium.com/@hiren6997/dependency-injection-wars-koin-vs-hilt-in-modern-android-projects-60ab6320b613>
12. Hilt vs Koin: The Hidden Cost of Runtime Injection (and Why Compile-Time DI Wins), accesso eseguito il giorno gennaio 18, 2026, <https://proandroiddev.com/hilt-vs-koin-the-hidden-cost-of-runtime-injection-and-why-compile-time-di-wins-3d8c522a073b>

13. Making your Android project modular with convention plugins | Michigan Software Labs, accesso eseguito il giorno gennaio 18, 2026,
<https://michiganlabs.com/blogs/making-your-android-project-modular-with-convention-plugins>
14. Clean Architecture: The ONE Change That Transforms Use Cases From Terrible to Super Useful - YouTube, accesso eseguito il giorno gennaio 18, 2026,
<https://www.youtube.com/watch?v=xrxC5gRNV48>
15. Level up your apps with the latest features from Android Health - Android Developers Blog, accesso eseguito il giorno gennaio 18, 2026,
<https://android-developers.googleblog.com/2024/05/the-latest-updates-from-an-droid-health-io-2024.html>
16. AI Vision on Android: CameraX ImageAnalysis + MediaPipe + Compose - ProAndroidDev, accesso eseguito il giorno gennaio 18, 2026,
<https://proandroiddev.com/ai-vision-on-android-camerax-imageanalysis-mediapipe-compose-db6afb738248>
17. Android UsageStatsManager - Tracking App Usage with Ease - CIIT Training, accesso eseguito il giorno gennaio 18, 2026,
<https://ciit-training.com/en/2024/09/16/android-usagestatsmanager-tracking-app-usage-with-ease/>
18. Gemini Nano Multimodal Capabilities on Pixel Phones - Google Store, accesso eseguito il giorno gennaio 18, 2026,
<https://store.google.com/intl/en/ideas/articles/gemini-nano-offline/>
19. Examples of qualifying software as a medical device under the EU MDR - Spyrosoft, accesso eseguito il giorno gennaio 18, 2026,
<https://spyro-soft.com/blog/healthcare/software-as-a-medical-device-examples>
20. Camera2 overview | Android media, accesso eseguito il giorno gennaio 18, 2026,
<https://developer.android.com/media/camera/camera2>
21. How to increase frame rate with Android CameraX ImageAnalysis? - Stack Overflow, accesso eseguito il giorno gennaio 18, 2026,
<https://stackoverflow.com/questions/57485050/how-to-increase-frame-rate-with-android-camerax-imageanalysis>
22. Configuration options | Android media - Android Developers, accesso eseguito il giorno gennaio 18, 2026,
<https://developer.android.com/media/camera/camerax/configuration>
23. Health Connect | Android health & fitness - Android Developers, accesso eseguito il giorno gennaio 18, 2026,
<https://developer.android.com/health-and-fitness/health-connect>
24. ADR process - AWS Prescriptive Guidance, accesso eseguito il giorno gennaio 18, 2026,
<https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html>
25. Trunk-based Development | Atlassian, accesso eseguito il giorno gennaio 18, 2026,
<https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development>

26. How to Use Feature Flags for Trunk-Based Development - Flagsmith, accesso eseguito il giorno gennaio 18, 2026,
<https://www.flagsmith.com/blog/trunk-based-development-feature-flags>
27. The 2026 Blueprint for Unbeatable Mobile App Architecture - Impact Techlab, accesso eseguito il giorno gennaio 18, 2026,
<https://impacttechlab.com/future-proof-your-app-the-2026-blueprint-for-unbeatable-mobile-app-architecture/>